Laboratory Work No. 7: Shortest Path Algorithms in Graphs

Course: Computer & Discrete Mathematics (CDM) | VNTU

Author: Student Yaroslav Sych | Vinnytsia National Technical University

1. Objective & Goal

Study Dijkstra's algorithm for finding shortest paths in weighted graphs, topological sorting, path reconstruction, and visual highlight of optimal routes.

2. Theoretical Background & Dijkstra Algorithm

Given a weighted graph G = (V, E, w) with non-negative edge weights w(e) >= 0 and a source vertex s:

Dijkstra's Algorithm Steps:

- Step 1: Assign tentative distance values d(v) = ∞ for all nodes, d(s) = 0. Mark all nodes unvisited.
- Step 2: For current node v, evaluate unvisited neighbors u and compute tentative distance d(u) = min(d(u), d(v) + w(v,u)).
- Step 3: Mark current node v visited. Select unvisited node with smallest tentative distance as new current node.
- Step 4: Repeat until target node is visited or unvisited set is empty. Time complexity is O(|V|^2) or O(|E| log |V|) with priority queues.

3. Pathfinding Visualizations

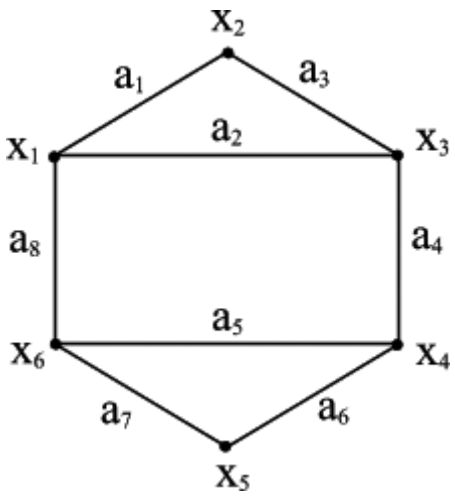

Figure: Weighted Graph Layout

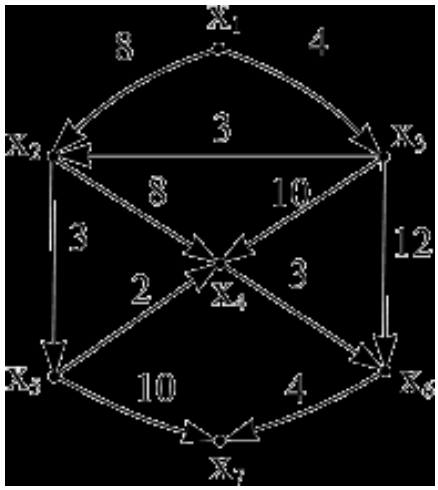

Figure: Shortest Path Output Highlight

4. Implementation Architecture

Defined in GraphPathFinder class using NetworkX and Matplotlib:

- topological_sort(): Checks topological order.
- Dijkstra pathfinder highlights calculated minimum cost path in bright red on graph layout.